COS344

Homework Assignment

Miniature Golf Course

Team Members:

Frikkie Malan (14439141)

Zaman Bassa (24744931)

Abdul Sabah (24566170)

Abdelrahman Ahmed (24898008)

Swelihle Makhathini (24584585)

Course Selected: Adventure Golf (Fourways, South Africa)

Date: 30 April 2026

1. Introduction

This report presents the proposed design and implementation plan for a real-time 3D rendering of the Adventure Golf Fourways miniature golf course[1] in Gauteng, South Africa, using OpenGL 3.3 and C++11. The objective of the assignment is to recreate an accurate and visually convincing 18-hole miniature golf environment based on a real-world course while maintaining interactive real-time performance.

The planned render will include realistic mixed-terrain, obstacles, decorative elements, a river, mixed lighting, shadows, and material surfaces inspired by the selected venue. A controllable drone camera system will allow users to explore the environment freely through the course using different viewpoints and movement controls. Special emphasis will be placed on implementing both day and night lighting modes, enabling the scene to transition between natural daylight and illuminated evening gameplay conditions (as is possible in the actual course). This report therefore serves as documentation that guides the modelling, rendering, interaction, and software architecture stages of the final implementation.

2. Background

2.1 Course Identification

Name: Adventure Golf (Fourways branch)
Location: Fourways, Gauteng (Ruby Cl, Witkoppen, Sandton, 2068)
Designer: Bill Lombard (owner) and Patrick (surname unknown)
Opening date: 2001
Course type: Outdoor
Theme style: Adventure and landscaped leisure environment. Also contains a tropical theme with excess shrubs and palm trees.
Playable conditions: Daytime and nighttime operation
Relevant facilities: Restaurant, entertainment areas, family recreation spaces
Operating Hours: 09:00 – 21:00 (Mon-Thur); 09:00 – 22:00 (Fri, Sat); 09:00 – 19:00 (Sun)

2.2 Brief History

Adventure Golf was established by the Lombard family 30 years ago in Gauteng, South Africa. It began as a small family-focused mini golf attraction has grown into a well known attraction with multiple branches. Adventure Golf has remained committed to providing memorable experiences for visitors.

2.3 Course Features Checklist

The selected course was evaluated against the assignment requirements and is considered suitable for implementation:

Requirement	Present
At least 18 holes	Yes
Incline/slope terrain	Yes
Water obstacle (dam/river/pond)	Yes
Tunnel feature	Yes (as part of multiple games)
Minimum three obstacles	Yes
Five decorative elements	Yes
Playable at day and night	Yes

Additional features observed include landscaped vegetation, themed props (i.e. steel giraffe statue, artificial boulders), retaining walls, walkways, lighting fixtures, and varied surface materials, all of which provide realism potential (Figures 1-3 below).

Figure 1. One hole illustrating the tropical theme, different palm trees and other vegetation, different textures (paving blocks, cement, synthetic grass, artificial boulders), bridges from planks, lighting and signage. Image used with permission from source.[2]

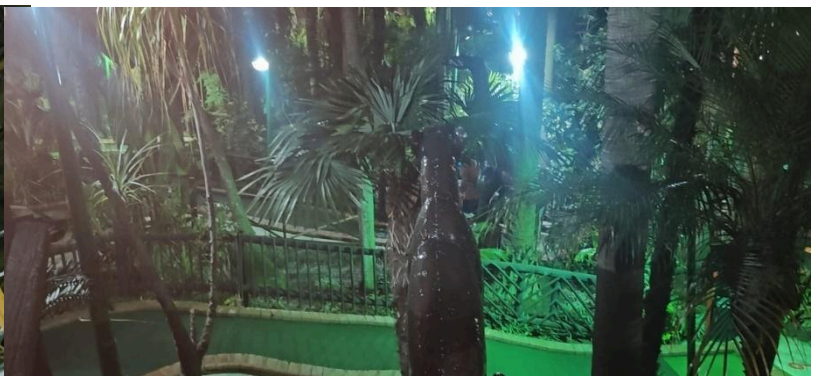

Figure 2. Pictures taken from the course (Photographer: Swelihle of the group) that illustrates the different plants present, the artificial boulders, the different slopes and steps (terrain and height changes), and lighting (evening) present.

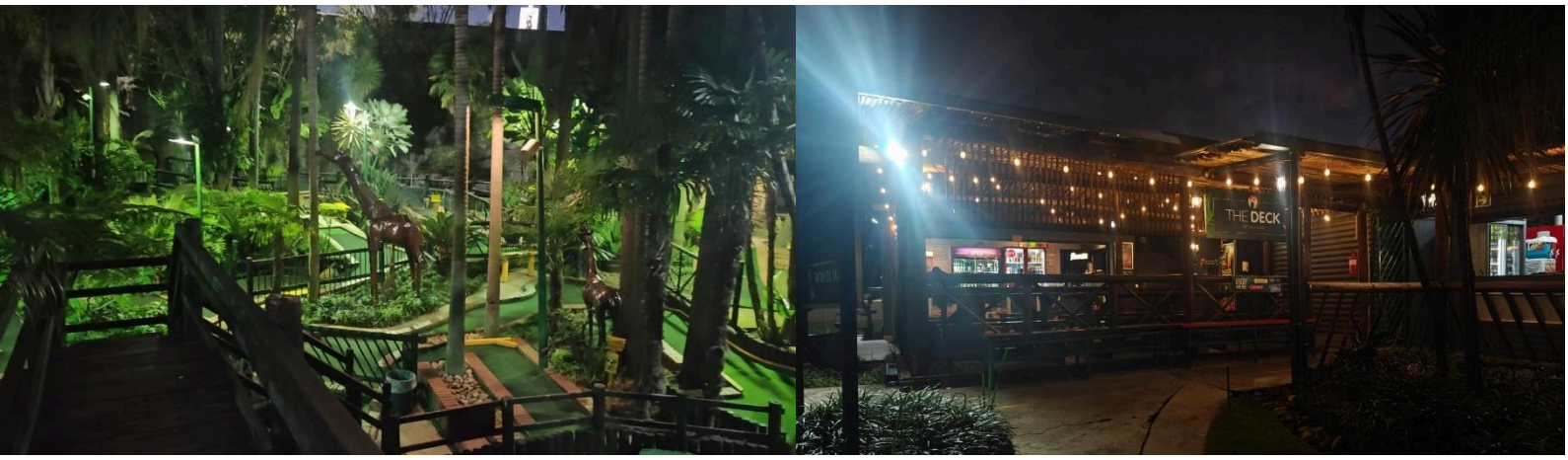

Figure 3. Pictures taken from the course (Photographer: Swelihle of the group) that illustrates the different terrain changes, the lighting, the giraffe statues, as well as the on-site restaurant.

2.4 Course Selection Rationale

Adventure Golf Fourways was selected because it offers an excellent balance between visual complexity, technical feasibility, and environmental realism. The course contains diverse hole layouts, decorative landscaping, water elements, terrain variation, and themed obstacles that create strong opportunities for modelling and shading. From a technical perspective, the course can be decomposed into modular repeated components such as walls, pathways, ramps, holes, fences, trees, rocks, and lighting fixtures, making it practical to construct efficiently in OpenGL using reusable geometry.

The outdoor setting is especially valuable because it enables implementation of both daylight and night-time rendering modes, dynamic shadows, and atmospheric skybox transitions. Furthermore, the family entertainment theme allows creative freedom when recreating decorative objects while still presenting the real-world reference course.

3. Miniature Golf Course Layout

3.1 Course Layout Overview

The Adventure Golf Fourways course will be modelled as an 18-hole miniature golf environment arranged within a bounded outdoor landscaped area. Based on reference images and site photographs, the course consists of interconnected walkways, themed hole zones, clusters of plants, decorative structures, and a river-like water feature. For implementation purposes, the environment will be mapped onto a Cartesian coordinate system in OpenGL world space. The global origin (0,0,0) will be positioned near the

geometric centre of the course, allowing hole layouts to extend in positive and negative X/Z directions. The Y-axis will represent elevation, enabling ramps, slopes, bridges, and raised terrain sections. A scaled top-down layout map and hole layouts are included in Figures 4 and 5, indicating approximate hole positions, pathways, vegetation zones, water areas, and decorative landmarks. The estimated total playable footprint of the course is approximately 100 × 100 world units, enclosed by decorative borders and surrounded by the skybox environment.

Figure 4. Top-view of the outline and perimeter of the course. Measurements are shown in the inset on the right hand side. The scale is shown on the bottom right (1:20 m).

Figure 5. Top-view of the course with approximate outlays of the 18 holes.

3.2 Hole Layout Strategy

In the table below, a proposal for the layout of each hole is made to accommodate varying levels of difficulty and terrain changes.

Hole	Features	Obstacles	Terrain	Notes
1	Straight starter	Bridge + side walls	Flat grass	Introductory hole
2	Figure-8 path	Triangular deflectors	Grass + cement	Multi-route
3	Gentle curve	Rock cluster	Slight incline	Requires angle shot
4	Tunnel hole	Tunnel + barriers	Flat	Precision aim
5	S-bend	Palm island centre	Grass	Rebound play
6	Split path	Two ramps	Mixed surfaces	Risk/reward
7	Long straight	Windmill	Flat	Timing challenge
8	Elevated bridge	Bridge over water	Incline + decline	Signature hole
9	U-shape	Corner bumpers	Grass	Bank shots
10	Spiral path	Inner rock obstacle	Flat	Accuracy
11	Raised tee	Ramp + tunnel	Multi-level	Complex geometry
12	River crossing	Narrow bridge	Water hazard	Precision
13	Zig-zag lane	Side barriers	Flat cement	Fast roll
14	Bowl hole	Central depression	Sloped	Gravity influence
15	Double curve	Decorative trees	Grass	Mid-difficulty
16	Narrow corridor	Columns	Flat	Precision
17	Island green	Water perimeter	Small incline	High pressure
18	Final challenge	Multi-obstacle + lights	Mixed terrain	Showcase hole

3.3 Object Construction Strategy

Each object in the course is built from one or more geometric primitives. Every primitive is uploaded as a VAO containing vertex positions (vec3), surface normals (vec3) for lighting, and UV coordinates (vec2) for texturing.[4] Triangles are indexed *via* an EBO. The table below lists each object, its constituent shapes, and the key rendering details.

For the cylinder construction: A cylinder with slices (lateral segments) may be built with (slices+1) vertices per ring (top and bottom). Each pair of adjacent columns generates two triangles. Normals point radially outward. A matching disk cap is generated separately so the normals there point along the axis. For the transformation pipeline: Every object may be positioned using a chain worldModel × localTranslate × localRotate × localScale. These are concatenated as 4×4 matrices before being passed as uMVP = proj × view × model and uModel = model to the shader. The normal matrix is mat3(transpose(inverse(uModel))) computed in the vertex shader, correcting non-uniform scaling.

Object	Primitive(s)	Approx. vertices/triangles	Vertex attributes	Shader/Material
Tunnel	2× cylinder (caps) + hollow cylinder body	~300 / ~500	pos, normal, UV	Diffuse + ambient; concrete texture
Ramp / slope	Wedge prism (5 faces: 2 triangles, 3 quads)	20 / 10	pos, normal, UV	Diffuse; grass or rubber texture
Borders / boundaries	Cuboid (6 quads)	24 / 12	pos, normal, UV	Diffuse + specular; painted concrete
Flag pole	Thin cylinder (radius ≈ 0.02)	~64 / ~64	pos, normal	Flat colour, specular (metal)
Flag	Single triangle mesh or 2-triangle quad	3–4 / 2	pos, normal, UV	Solid colour or texture; alpha-blended
Golf hole	Disk (filled circle, ~32 segments)	~34 / ~32	pos, normal	Solid black, no lighting
Lamp post	Cylinder (pole) + sphere (bulb)	~150	pos, normal	Pole: specular metal; bulb: emissive
Rock decoration	Low-poly icosphere (2 subdivisions)	~80 / ~160	pos, normal	Diffuse; grey noise texture
Tree trunk	Cylinder	~64 / ~64	pos, normal, UV	Diffuse; bark texture
Tree canopy	Cone or sphere	~100	pos, normal, UV	Diffuse; green noise texture
Windmill body	Cylinder + cuboid base	~200	pos, normal, UV	Diffuse; wood/brick texture
Windmill blades	4× flat quads rotated 90° apart	16 / 8	pos, normal, UV	Diffuse; wood texture
Water surface	Subdivided plane (NxN quads)	N^2 / $2N^2$	pos, normal, UV	Animated water shader
Sky box	6 inward-facing quads	24 / 12	pos, UV	Texture only, depth-clamped

3.4 Example OpenGL Hole

To demonstrate the intended modelling approach for the final miniature golf course, an example hole rendered in OpenGL 3.3 from a previous practical assignment is included below (Figure 6). This example illustrates how a complete golf hole can be constructed from simple geometric primitives combined through transformations such as translation, scaling, and rotation. The purpose of this prototype is to show how complex course layouts can be assembled efficiently using reusable shapes while maintaining real-time rendering performance.

The hole consists of several key geometric components. The starting area is represented using a flat square surface positioned at the starting point of the hole. The main fairway is modelled as a subdivided plane that forms the playable putting surface and is scaled to the desired dimensions. The target hole cup is positioned near the upper right corner of the hole and is represented using a circular disk. Border walls are constructed from elongated cuboids placed around the perimeter of the hole to guide gameplay and prevent the ball from leaving the surface. Additional obstacle elements are included using cuboids or cylinders (variable number of vertices) to increase difficulty. A flagpole and flag are positioned at the hole cup using a vertical cylindrical pole with a rectangular flag attached.

Figure 6. Scene of Frikkie Malan's (group member) practical 2 Render.

The scene is assembled using standard model transformations. The fairway plane lies at ground level and then scaled to the required width and length of the hole. Border walls are translated to the edges of the fairway and scaled along either the X-axis or Z-axis depending on their orientation. Obstacles (and the ball) may be translated to specific gameplay positions and resized according to their intended proportions. The hole cup may be translated to its final target location, while the flagpole is translated to the same position and scaled vertically to achieve realistic height. Where necessary, selected objects may also be rotated to align with angled pathways or alternative layouts. Collision detection of the ball with the obstacles was also incorporated in this practical, adding to the realism.

4. Skybox Design

4.1 Skybox Dimensions

The skybox will be implemented as a cube, but will be surrounding the entire golf course, this will then ensure that the environment will look infinite. We will implement the skybox much larger than the course box; this will help us to prevent the camera from reaching the edges. For example, if the skybox is 300 x 300 x 300 units, then the course will be smaller, so will be approximately 100 x 100 units. The skybox is centered at the origin of the world (0,0,0) and will move with the camera position to maintain the illusion of distance. Depth testing will be adjusted (rendered last or with depth trick) so that the skybox always appears behind all objects**.**

4.2 Skybox Texture Strategy

The skybox will be using a cube mapping, where there will be 6 textures that will represent all the faces of the cube (top, bottom, left, right, front and back). We will be using two types of textures, which are the following:

- ➔ For the day skybox: for the bright sky, clouds and sunlight
- ➔ For the night skybox: For the stars and darker tones.
- ➔ For transitioning between the day and night (ie dusk and dawn), warm orange/yellow/pink tones are incorporated

We will have references to images, to provide more clarity and to improve the visual clarity. We will ensure that the horizon line will line up (to the best of our ability) to ensure that all the sides in the cube are joined. Reference images (Figure 7) of the resultant render are provided below.

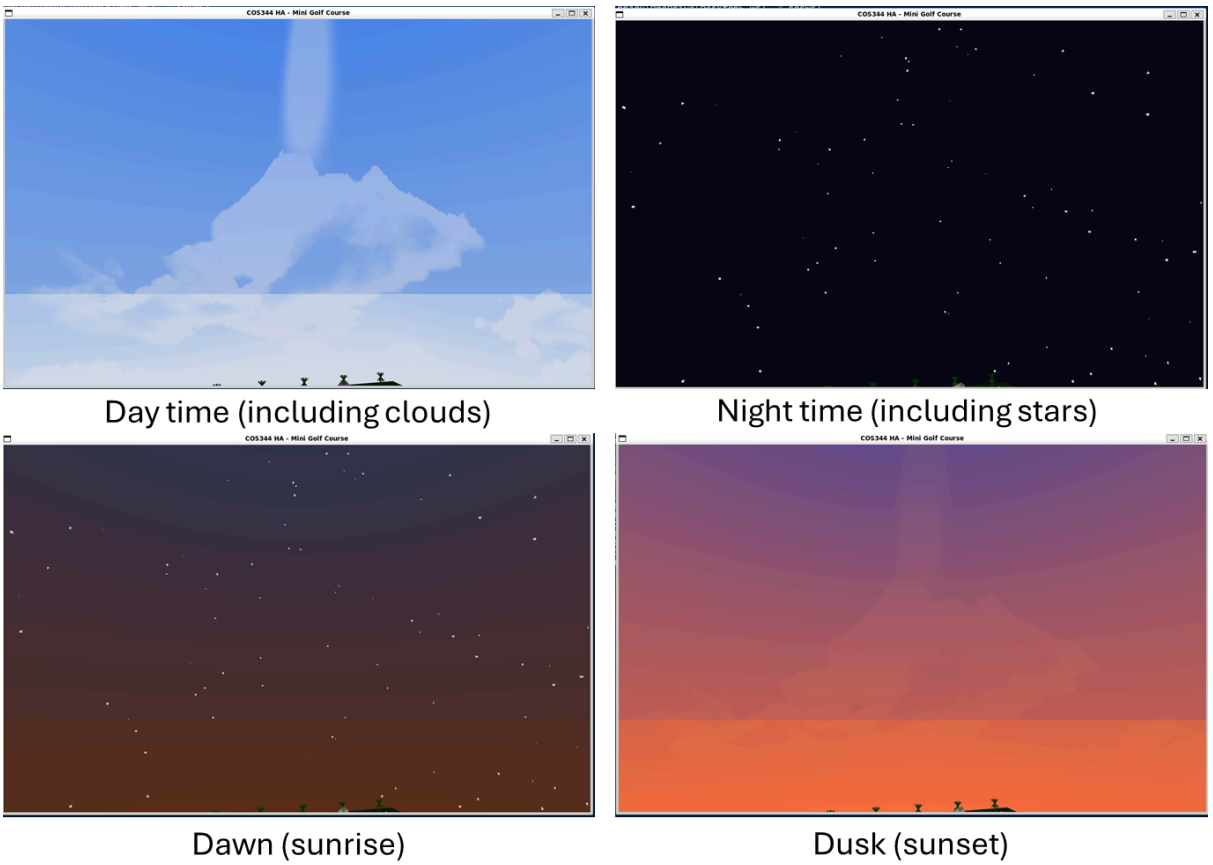

Figure 7. The skybox during different times of the day.

5. Terrain Design

5.1 Terrain Height Variation

Slopes and ramps are implemented at the mesh level rather than through a height map, because the selected miniature golf terrain is composed of well-defined geometric surfaces rather than organic landscape. We may consider two techniques to achieve this:

(i) Mesh tilting (rigid inclines): A flat grid is rotated about its local X- or Z-axis by the desired incline angle before being positioned in the world. This is sufficient for straight ramps. The normal vectors, derived from the rotation, automatically tilt to face the correct direction for lighting.

(ii) Vertex displacement (gradual slopes): For smoother "natural" slopes (e.g., a banked curve), the y-coordinate of vertices in a subdivided plane is set according to a mathematical function (linear or quadratic) of the horizontal distance. More grid subdivisions produce a smoother curve with fewer lighting artefacts.

The fairway mesh at each hole is modelled with the appropriate combination of these techniques to match the reference course layout.[5,6]

5.2 Terrain Surface Types

The table below summarises each terrain type, its rendering method, and the key visual properties. To better illustrate the differences, Figure 8 shows all six types rendered on 1×1 unit tiles under a single point light.

Terrain	Rendering method	Key shader technique	Typical colours
Grass	Diffuse + ambient, value noise	Noise modulates green hue + fine blade pattern	(0.12, 0.42, 0.09) base

Sand	Diffuse + bump mapping	Noise gradient perturbs surface normal to simulate grain texture	(0.88, 0.78, 0.50) base
Cement	Diffuse + specular (Phong)	Low-frequency noise for subtle surface variation; moderate Ks	(0.65, 0.65, 0.67)
Paving blocks	Diffuse + specular	UV modulo creates grout lines; per-cell hash drives tile shade variation	Mixed warm grey
Water	Diffuse + high specular + animated normals	Multi-frequency sine sum drives normal map; large specular highlight (n=128)	(0.05, 0.22, 0.62)
Gravel	Diffuse, hash noise	Per-stone coarse hash + fine value noise	Speckled grey-brown

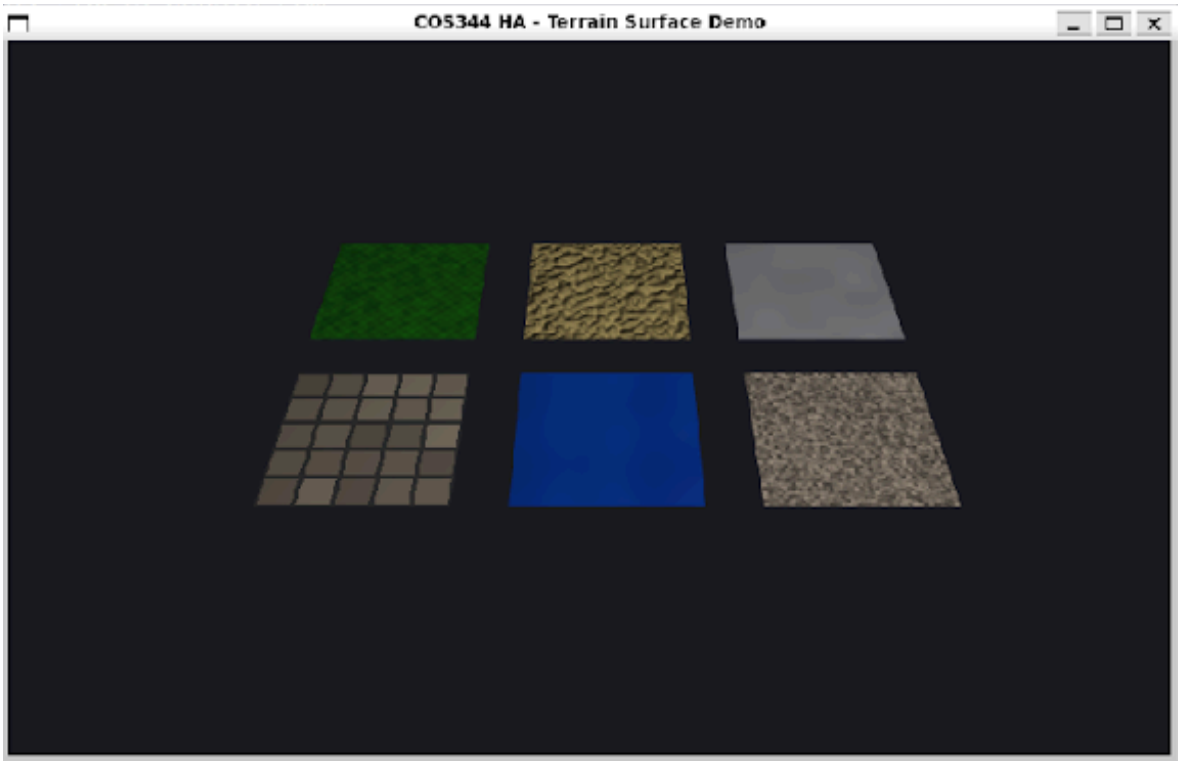

Figure 8. Render of six different terrains (from top left to right: grass, gravel, metal, paving blocks, water, and sand) under ambient lighting. The water tile is visibly animated in the live program.

6. Obstacles

6.1 Course Obstacles

Each obstacle below is described with its primitive breakdown, key transformations, and approximate dimensions. Tunnels are represented by hollow cylinders. The floor is a matching rectangle at ground level. Players must aim through the opening. Rendered with a concrete diffuse texture.

Ramps are represented as wedge prisms with the inclined face having a grass texture and faces the tee. The underside of ramps have a plain concrete texture. Ramps are resized to fit course dimensions and rotated about the x-axis to create the slope.

Barriers and side walls are represented with elongated cuboids along the edge of the fairway. Some are angled to redirect the ball.

For some of the decorative items, the following may be considered in the table below.

Prop	Shape	Texture/Colour
Palm tree	Cylinder trunk + cone canopy	Bark noise / green noise
Bench	3× cuboids (seat + 2 legs)	Wood texture
Rock cluster	Low-poly sphere (scaled flat)	Grey noise
Fountain	Cylinder + torus (ring)	Concrete + animated water
Signage	Cylinder pole + flat quad	Metal + text texture

6.2 Windmill Integration

The windmill from Practical 3 (group member Frikkie Malan) is incorporated as a course obstacle on hole 7, positioned at the midpoint of a straight fairway section. The windmill's rotating blades are timed such that a correctly struck ball can pass through the gap between blade sweeps, adding a skill element (Figure 9).

Hole 7 is the longest straight on the course. The windmill provides a visual centrepiece and a timing challenge without requiring the player to change direction. The gap at the bottom of

the blade rotation is wide enough for the ball to pass when timed correctly. Hence, it would make sense to incorporate the windmill as part of Hole 7. Integration: The windmill is rendered as a sub-scene which consists of the body (cuboid base) is positioned with translate(hole7_x, 0, hole7_z). The blade group is rotated each frame: rotate(bladeAngle, 0, 0, 1) where bladeAngle += BLADE_SPEED * dt. The windmill shader shall receive the same light sources as the rest of the scene.

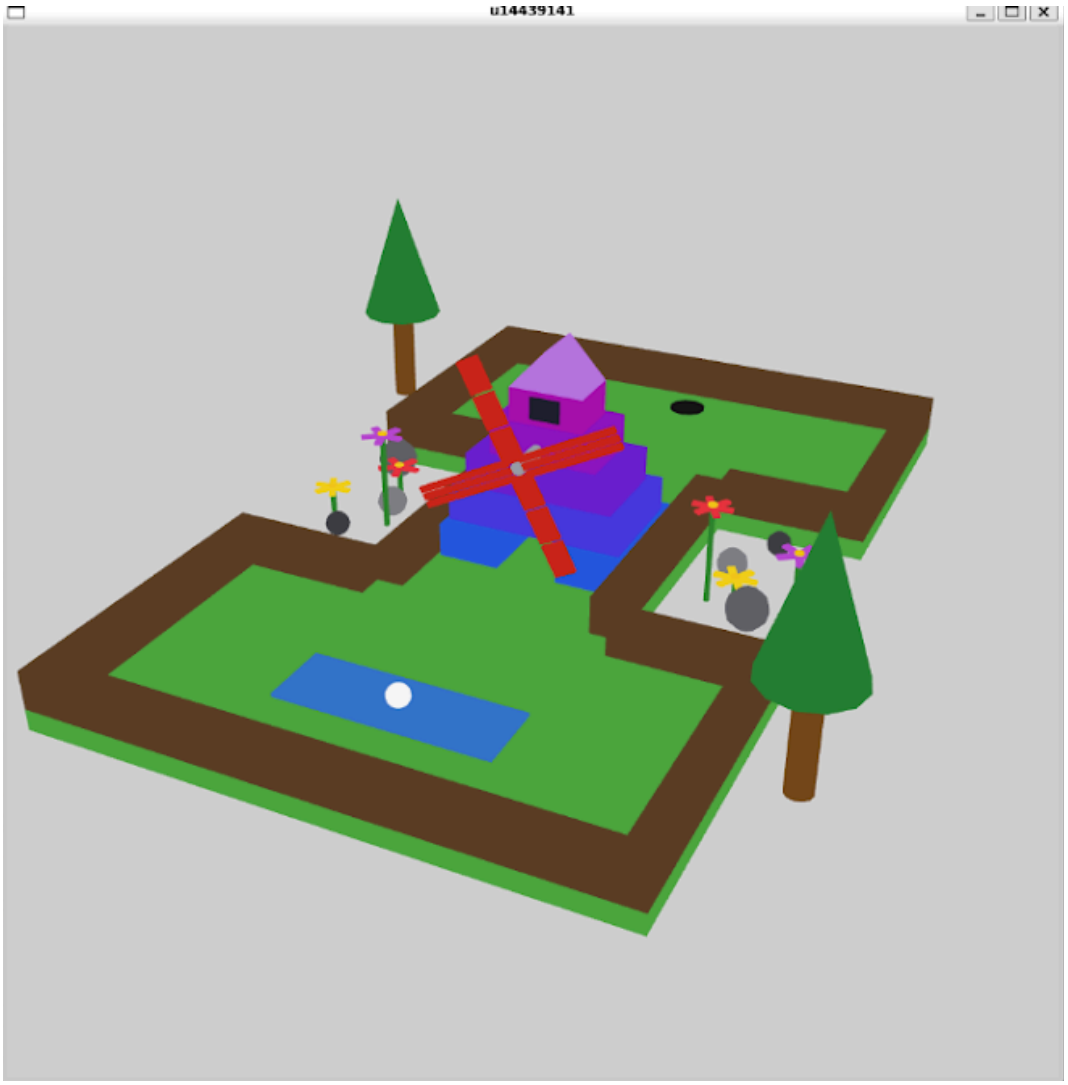

Figure 9. Windmill render from Practical 3 (Frikkie Malan), showing the vertical grid-like blades that passes just above the grass terrain. As part of the realism aspect in this assignment, more cuboids could be stacked more densely for a “smoother” effect of stacked planks.

7. Materials

All materials will use the Phong Illumination Model.[7,8] This model approximates ambient, diffuse, and specular light interaction while remaining efficient for real-time OpenGL rendering. Accordingly, the reflected light at a surface point is:

$$I = \mathbf{Ka}\cdot\mathbf{Ia} + \mathbf{Kd}\cdot(\mathbf{L}\cdot\mathbf{N})\cdot\mathbf{Id} + \mathbf{Ks}\cdot(\mathbf{R}\cdot\mathbf{V})^{\mathbf{n}\cdot\mathbf{Is}}$$

Where: **Ka** = ambient coefficient (surface colour under indirect light); **Kd** = diffuse coefficient (Lambertian response, depends on angle **L·N**); **Ks** = specular coefficient (mirror-like highlight, depends on **R·V** angle raised to shininess n); **Ia**, **Id**, **Is** = ambient, diffuse and specular intensities of the light source, respectively; **L** = unit vector from fragment to light; **N** = surface normal; **R** = reflection of **L**; **V** = unit vector to (drone) camera.

In this model, point-light attenuation multiplies the diffuse and specular terms:

$$att = 1 / (\mathbf{Kc} + \mathbf{Kl}\cdot\mathbf{d} + \mathbf{Kq}\cdot\mathbf{d}^2)$$

The table below summarises per-material Phong coefficients:

Material	Ka	Kd	Ks	n (shininess)	Notes
Grass	0.22	0.80	0.00	—	No specular; noise-driven colour
Sand	0.22	0.75	0.00	—	Bump-mapped normals
Cement	0.22	0.70	0.35	64	Slight sheen
Paving	0.22	0.75	0.20	48	Per-tile variation
Water	0.22	0.50	0.90	128	Animated normal, high gloss
Gravel	0.22	0.80	0.00	—	Hash noise colour
Metal (pole)	0.15	0.60	0.70	96	High specular
Wood	0.20	0.80	0.10	32	Textured, low gloss
Polished ball	0.10	0.40	0.90	128	Semi-transparent
Rock	0.22	0.70	0.12	18	Rough surface, soft specular

Textures are to be generated procedurally in the GLSL fragment shader. Value noise and hash functions will be used to avoid needing to load external image files. For the grass and wood surfaces, the noise is used to modulate hue. For sand, the noise gradient is used to perturb the surface normal (i.e. bump mapping). A selected few actual reference photos are included in Figures 10 and 11 below to provide more context of the different textures and colours associated with specific elements that are to be rendered.



Figure 10. Reference terrain and surface materials present on the course, including synthetic grass, paving blocks, plants, lighting poles, signage, and walkway surfaces used for terrain rendering calibration.

Figure 11. Reference material textures observed at Adventure Golf Fourways, including wood, stone/boulders, coloured polished balls, water, paving bricks, concrete, and decorative structural finishes used to guide procedural shader design.

8. Lighting Design

8.1 Light Sources

We will use a directional light source which will be used to represent the sun and provide lighting in the course. We will use point lights that will be used as lamps to light specific areas in the course. We will have a spotlight on the drone to show that the drone is moving. To improve the realism, we will have different light sources with different intensities. The colours would be warm yellow (2700 K and 3000 K)

Three categories of light source are to be used:

(i) Directional light (sun/moon): A directional (ambient) light has no position and therefore sends parallel rays from a fixed direction vector. In the shader it may be computed as:

```
vec3 lightDir = normalize(-uSunDir);
float diff = max(dot(norm, lightDir), 0.0);
```

The sun direction rotates slowly as the day/night cycle progresses.

(ii) Point lights (lamp posts): Each lamp post contributes a point light with uLightPos in world space and quadratic attenuation $1/(Kc + Kl \cdot d + Kq \cdot d^2)$. The course has ~12 lamp posts positioned along the walkways.

(iii) Spotlight (drone camera light): The drone carries a spotlight modelled as a point light with a cone cutoff:

```
float theta = dot(lightDir, normalize(-uSpotDir));
float epsilon = uCutoffInner - uCutoffOuter;
float intensity = clamp((theta - uCutoffOuter) / epsilon, 0.0, 1.0);
```

8.2 Day/Night Mode Strategy

In our course we will be able to switch from day to night and from night to day, by pressing a button. In the day mode:

- We will have directional sunlight that will be strong and visible
- It will be a very bright day

For the night mode:

- The sunlight will be reduced
- Both the lamps and drones will become more visible

When switching between these modes, we will also change both the colours and intensity. The proposal is for a uniform uTimeOfDay (0.0 = midnight, 0.5 = noon) that drives three changes:

(i) Sun direction: sunDir = vec3(cos(2π·t), sin(2π·t), 0.3). This allows the light to sweep from horizon to horizon.

(ii) Sky box texture swap: Two sky box textures (day and night) are to be blended during dawn/dusk using a mix(dayTex, nightTex, twilightFactor).
(iii) Lamp post lights: Activated when uTimeOfDay < 0.25 || uTimeOfDay > 0.75 (night time, 18:00 to 06:00). During the day the lamp lights contribute near-zero intensity.

We suggest a keyboard shortcut (e.g., +) that increments uTimeOfDay to allow the control of stepping through the cycle.

8.3 Shadows

Shadows will be incorporated to improve realism, depth perception, and overall visual quality within the miniature golf course environment. Two complementary shadow techniques are proposed: shadow mapping for dynamic light-based shadows, and projection shadows for efficient planar shadows cast onto fairway surfaces. Shadow mapping will be used for major scene lighting sources, as it allows shadows to be generated according to the position and direction of the light source. This produces realistic moving shadows from objects such as bridges, trees, flags, walls, and decorative props as lighting conditions change. Shadow projection is to be implemented using projection shadow matrices (simple planar shadows) for all course objects onto the fairway planes and course. For each directional light source with direction **L** and a receiving plane at y = 0:

$$M_{shadow} = (\mathbf{N}\cdot\mathbf{L})\cdot\mathbf{I} - \text{outer\_product}(\mathbf{L}, \mathbf{N})$$

Where **N** = (0,1,0,0) is the plane normal and **I** is the identity. This projects each vertex onto y = 0 along **L**. The shadowed geometry is rendered in a second pass as a dark semi-transparent overlay (alpha ≈ 0.5, depth-offset to avoid z-fighting).

Reflections may also be incorporated to improve realism during the transition between day and night. We propose simple reflective lighting effects on selected surfaces (such as water or glossy terrain elements) by using fragment shader calculations that respond to the active light sources and the current uTimeOfDay value. This will enhance the atmosphere of the scene without introducing excessive rendering complexity.

9. Drone Camera System

A controllable drone camera system will be implemented to allow the user to freely explore the mini-golf course environment from multiple viewpoints. The idea is that the drone provides full 3D movement through the rendered scene, enabling inspection of hole layouts, obstacles, lighting effects, terrain variation, and decorative elements. The drone then acts as the primary interaction mechanism between the user and the virtual course and is therefore an important component of the final experience. Additional camera functionality will include the ability to switch between perspective and orthographic projection modes during runtime. This will be implemented using the Strategy Design Pattern, where different

projection calculation strategies can be selected and applied to the drone camera system without modifying the main rendering pipeline. A keyboard shortcut (P) is used to toggle between orthographic and perspective views while navigating the course. To reduce the possibility of gimbal locking during drone rotation, the camera orientation system will make use of constrained pitch rotation together with recalculated direction vectors derived from yaw and pitch values.

9.1 Drone Movement Controls

The drone will support smooth six-degree-of-freedom movement through the scene. Translational movement will allow motion forward, backward, left, right, upward, and downward relative to the drone's local orientation. Depending on time constraints, rotational control will allow yaw and pitch adjustments so that the user can look around naturally while navigating. Drone movements will be controlled as follows:

Key	Action
W / S	Move forward / backward
A / D	Move left / right
Q / E	Move upward / downward
Arrow keys / Mouse	Adjust pitch and yaw

9.2 Camera Projection

The drone camera will be implemented using standard view matrix techniques. Its position in world space will be stored as a 3D vector, while orientation will be represented using forward, right, and up direction vectors derived from yaw and pitch angles. At each frame, the camera view matrix will be recalculated using a lookAt() transformation so that all rendered geometry is viewed relative to the drone's current pose. This allows for free-flight navigation while maintaining compatibility with the OpenGL rendering pipeline. To ensure smooth motion independent of frame rate, it is considered to have movement updates be scaled using delta time between frames.

9.3 Drone Features

The drone may include features such as the following:

- The lights that will be visible
- Collision detection to prevent crashes
- We will have a night vision for the low-light environments

10. Code Architecture

A modular and object-oriented software architecture is proposed for the implementation of the miniature golf course renderer. The primary objective of the code structure is to maintain clear separation of responsibilities, encourage reuse of common functionality, and simplify future expansion as additional gameplay and bonus features are introduced. By dividing the program into focused classes and source files, the project becomes easier to test, debug, and maintain.

10.1 Proposed file Structure

A separation of concerns approach will be followed, where class declarations are placed in header files and functionality is implemented in the corresponding source files. Rendering logic, geometry generation, lighting systems, and scene management will therefore remain logically independent. The expected file structure is outlined below:

- main.cpp: application entry point, window creation, render loop, input handling, and high-level scene control.
- shader.hpp / shader.cpp: shader compilation, linking, uniform management, and GPU program switching.
- camera.cpp / drone.cpp: drone movement, camera orientation, projection handling, and navigation controls.
- course.cpp: manages the overall golf course layout, object placement, and scene composition.
- terrain.cpp: generation of fairways, ramps, inclines, slopes, rivers, and surface meshes.
- lighting.cpp: directional lights, point lights, attenuation, shadows, and day/night switching.
- obstacles.cpp: bridges, tunnels, barriers, windmill, props, and interactive objects.
- skybox.cpp: skybox geometry, texture loading, and atmosphere transitions.
- shape2D.cpp / shape3D.cpp: reusable primitive geometry generation such as circles, squares, cubes, cylinders, prisms, and spheres.
- materials.cpp: Phong material parameters and surface presets for grass, cement, water, wood, and metal.
- utils.cpp: helper mathematics, collision routines, and common utility functions.

10.2 Data Structures Used

A range of standard C++ data structures will be used to organise scene content efficiently. Vectors (std::vector) will be heavily utilised for storing dynamic collections of objects such as holes, obstacles, lights, decorative assets, and generated vertex data. Vectors are well suited to this project because the number of scene objects may change during development and they provide contiguous memory storage for efficient iteration. Structs will be used for lightweight data containers where behaviour is minimal. Examples include vertex definitions

containing position, normal, and texture coordinates, material parameter sets, transformation data, or collision bounds. Structs simplify the passing of grouped rendering data between systems. Classes will form the foundation of the object-oriented design. Major scene entities such as the Course, Drone, SkyBox, Light, Terrain, and Shape hierarchies will be represented as classes containing both data and behaviour. Encapsulation will allow each component to manage its own state while exposing only the required public interface. Where appropriate, enumerations may also be used for options such as projection mode, light type, terrain category, or camera state. This improves readability and reduces the use of ambiguous constants.

10.3 Design Patterns Used

Several classical software design patterns are proposed to improve maintainability and reduce duplication within the rendering system, as is shown in the table below, as well as Figure 11.

Figure 11 presents a preliminary UML class diagram illustrating the proposed relationships between the major classes in the system. The diagram reflects the planned inheritance hierarchies for shapes and projections, aggregation relationships between the course and scene objects, and the use of selected design patterns such as Composite, Decorator, Strategy, and Singleton. This design remains provisional and may be refined during implementation as practical constraints and performance considerations emerge. However, it provides a strong structural foundation for the development of the final OpenGL render.

Pattern	Proposed usage	Justification
Composite	Course made up of holes, obstacles, decor, lights, and terrain elements	Allows complex scenes to be treated as collections of simpler components with a uniform interface
Singleton	ShaderManager	Only one global shader controller is required to manage compiled shader programs and uniforms
Decorator	Additional behaviour or state added to objects or course elements	Allows decorations, modifiers, or feature extensions to be layered dynamically without rewriting base classes
Template Method	Shape generation hierarchy	A base Shape class can define common creation steps while derived 2D and 3D shapes implement specific geometry generation
Strategy	Drone projection system	Orthographic and Perspective projections can be swapped at runtime through a common projection interface

11. Bonus Features Plan

Several optional features are planned to extend the project beyond the minimum assignment requirements and to improve both realism and interactivity. The primary objective is to transform the rendered environment from a static visual scene into a fully playable miniature golf experience.

A key planned feature is the inclusion of a controllable golf ball with realistic movement physics. The ball will respond to an initial strike force and move according to simulated velocity, momentum, and gradual frictional deceleration. Surface type will influence movement behaviour, with smoother materials such as cement allowing longer rolling distances, while artificial grass surfaces will generate greater resistance and reduce travel distance. Terrain gradients will also affect motion, with downhill slopes increasing ball speed and uphill slopes reducing momentum. This will add strategic gameplay and realism to the simulation. To support full playability, collision detection will be implemented between the ball and surrounding geometry such as walls, barriers, obstacles, tunnels, and decorative boundaries. Collisions will alter the direction of travel and prevent the ball from passing through solid objects. This builds on techniques previously explored in earlier practical work and can be extended to more complex obstacle interactions.

Another planned enhancement is the animation of flowing water features such as rivers or streams. This can be achieved through scrolling textures, normal perturbation, or vertex displacement to simulate movement across the water surface. Animated water would significantly improve realism and make environmental hazards feel more dynamic. A time-of-day system is also proposed, allowing the scene to transition gradually between daylight and night-time conditions. This would modify skybox textures, ambient lighting intensity, directional sunlight colour, and the activation of artificial light sources around the course. Such a feature would enhance the atmosphere while showcasing dynamic lighting techniques.

12. Implementation Timeline

The project will be approached in progressive stages, beginning with planning and core scene construction, followed by advanced rendering features, gameplay mechanics, and final polishing.

By 30 April 2026: Initial Report Submission
Complete the planning document, including course selection, layout analysis, modelling strategy, lighting plan, drone design, materials, terrain, and bonus feature proposals. Gather all reference images, maps, and measurements required for implementation.

1 May – 13 May 2026: Core Environment Development
Construct the base miniature golf course layout in OpenGL, including terrain surfaces, fairways, hole geometry, border walls, ramps, tunnels, water areas, and key decorative

objects. Implement reusable object classes and establish scene management structure. Develop the skybox and basic daytime lighting system.

By 13 May 2026: Peer Review Deadline
Submit peer reviews of other teams' reports. Use received feedback to improve planning decisions and identify weaknesses in the current implementation.

14 May – 20 May 2026: Advanced Rendering and Interaction
Implement night-time lighting mode, multiple light sources, shadows, improved materials, and animated environmental elements such as flowing water. Finalise drone camera controls, movement, rotation, and projection modes.

21 May – 24 May 2026: Gameplay and Bonus Features
Introduce the golf ball system with collision detection, ball physics, momentum, friction, and terrain response. Implement optional features such as time-of-day transitions, zoom controls, and other features where time permits. Conduct performance optimisation (if necessary) and bug fixing.

22 May 2026: Final Report Submission
Submit the refined final report incorporating peer feedback, updated screenshots, completed implementation details, and final design decisions.

25 May 2026: Program Submission and Demo Preparation
Finalise source code, ensure successful compilation, prepare the archive submission, and test controls thoroughly for demonstration readiness.

13. References

1. Adventure Golf website (Home). https://adventuregolf.co.za/. Date accessed: 27 April 2026.

2. Adventure Golf website (Fourways). https://adventuregolf.co.za/pages/fourways-branch/. Date accessed: 27 April 2026.

3. Adventure Golf website (Our Story). https://adventuregolf.co.za/pages/our-story/. Date accessed: 27 April 2026.

4. Joey de Vries. LearnOpenGL (Modern OpenGL Tutorial). https://learnopengl.com/. Date accessed: 27 April 2026.

5. Joey de Vries. LearnOpenGL (Transformations). https://learnopengl.com/Getting-started/Transformations. Date accessed: 27 April 2026.

6. Joey de Vries. LearnOpenGL (Lighting/Normals). https://learnopengl.com/Lighting/Basic-Lighting. Date accessed: 27 April 2026.

7. Joey de Vries. LearnOpenGL (Basic Lighting/Materials). https://learnopengl.com/Lighting/Materials. Date accessed: 27 April 2026.

8. Joey de Vries. LearnOpenGL (Light Casters). https://learnopengl.com/Lighting/Light-casters. Date accessed: 27 April 2026.